UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 Redes



Laboratorio 3 Algoritmos de Enrutamiento

Informe individual de implementación

Ihan Marroquin - 23108

GUATEMALA, 07 DE SEPTIEMBRE DE 2026

Informe individual

Repositorio de código: github.com/Ihan-Marroquin/lab3_redes

Este informe documenta la parte individual del Laboratorio 3 de CC3067 Redes. La solución implementada contiene Dijkstra, Flooding, Link State Routing y Distance Vector Routing sobre un protocolo JSON común. La verificación local confirmó la entrega de mensajes en los cuatro modos, la convergencia de los algoritmos dinámicos, la reacción ante cambios de enlace y el reenvío entre procesos independientes mediante sockets TCP. La prueba conjunta con el servidor XMPP se realizará en clase durante la entrega.

1 Introducción

Un router necesita decidir por cuál vecino debe reenviar un paquete para acercarlo a su destino. Cuando la red cambia, una tabla estática deja de representar los enlaces disponibles y puede producir rutas inválidas. Por ello, los algoritmos de enrutamiento distribuyen o calculan información que permite formar y actualizar las tablas sin depender de un conocimiento global permanente.

La práctica se resolvió como una biblioteca modular en Python 3.11. Cada algoritmo mantiene su propio estado y recibe únicamente la información permitida por el enunciado. Dijkstra puro consulta la topología completa; Flooding conoce sus vecinos; LSR reconstruye la topología a partir de anuncios de estado de enlace; y DVR aprende las distancias anunciadas por los vecinos. Esta separación permite probar los algoritmos en memoria y reutilizarlos con un transporte TCP.

2 Objetivos

- Implementar y comparar Dijkstra, Flooding, Link State Routing y Distance Vector Routing.
- Representar mensajes de datos y control mediante una estructura JSON interoperable.
- Mantener separados los procesos de forwarding y routing para ejecutar cada nodo de forma concurrente.
- Comprobar entrega, convergencia, control de ciclos, adaptación a fallos y comunicación TCP con pruebas reproducibles.

3 Descripción de la práctica

El laboratorio plantea una red de nodos en la que cada proceso puede originar, recibir o reenviar mensajes. Los nodos se inicializan con un archivo de topología y emplean paquetes de datos, información y descubrimiento. La primera fase usa sockets para comprobar los algoritmos localmente; la segunda sustituirá ese medio por el servidor XMPP proporcionado por el curso.

El alcance individual comprende el diseño del protocolo, los cuatro algoritmos, el simulador, la ejecución TCP, los archivos de ejemplo y las pruebas automatizadas. No se incluyen resultados de la prueba grupal ni del servidor XMPP porque esa validación se efectuará durante la sesión de entrega. El código deja el transporte desacoplado para incorporar esa conexión sin reescribir los routers.

4 Diseño e implementación

La solución divide la responsabilidad en cinco módulos. El protocolo valida los paquetes; la configuración carga topologías, nombres y endpoints; cada router calcula decisiones; el simulador ejecuta eventos deterministas; y el runtime mueve los paquetes por sockets. Esta distribución evita que un algoritmo dependa del medio de comunicación y permite probarlo aisladamente.

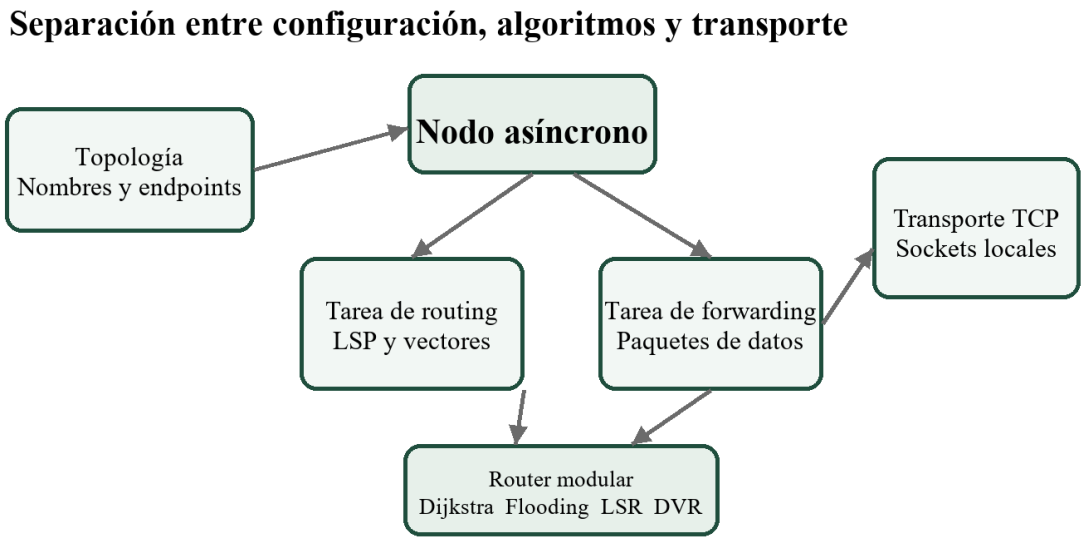


Figura 1. Arquitectura modular de la solución individual.

Componente	Responsabilidad	Archivo principal
Protocolo	Validar, serializar y reducir el TTL de paquetes JSON.	protocol.py
Algoritmos	Calcular rutas, propagar información y decidir el siguiente salto.	algorithms/
Simulador	Conectar routers en memoria y registrar trazas reproducibles.	simulator.py
Runtime TCP	Ejecutar forwarding y routing concurrentes entre procesos.	runtime.py
CLI	Iniciar simulaciones, consultar tablas y levantar nodos.	cli.py

Tabla 1. Responsabilidades de los módulos implementados.

4.1 Protocolo común

La clase Packet representa la estructura indicada por el laboratorio. Es inmutable y exige un protocolo soportado, un tipo válido, origen, destino, TTL no negativo, headers y payload. Cada mensaje recibe un packet_id único; al reenviarlo se crea una copia con TTL reducido, lo que evita alterar el objeto original y simplifica las pruebas.

Campo	Uso
proto	Selecciona dijkstra, flooding, lsr o dvr.
type	Distingue message, echo, info, hello o ack.

Campo	Uso
from y to	Identifican el origen y el destino final.
ttl	Limita la cantidad de saltos y detiene ciclos.
headers	Transporta packet_id y metadatos opcionales.
payload	Contiene el mensaje del usuario o el anuncio de routing.

Tabla 2. Campos del paquete JSON compartido.

4.2 Dijkstra

Dijkstra calcula el costo mínimo desde un origen hacia todos los destinos alcanzables. La implementación usa una cola de prioridad y conserva la ruta completa como criterio secundario, por lo que los empates se resuelven de manera determinista. La tabla resultante guarda destino, siguiente salto, costo y secuencia de nodos. También puede refrescarse cuando cambia la topología del modo estático permitido para Dijkstra puro (Cormen et al., 2022).

4.3 Flooding

Flooding reenvía cada paquete a todos los vecinos excepto al enlace por el que llegó. No necesita una tabla de routing, pero debe controlar el crecimiento de copias. La solución combina TTL, un packet_id y una caché LRU acotada de identificadores vistos. El destino entrega la primera copia y los duplicados posteriores se descartan. Esta estrategia garantiza alcance en una componente conectada, aunque no garantiza la ruta de menor costo y produce más transmisiones.

4.4 Link State Routing

Cada router LSR origina un Link State Packet con sus enlaces locales, costos y número de secuencia. Los vecinos propagan únicamente los anuncios nuevos mediante flooding. Con los registros recibidos, el nodo construye su propia base de estados y reutiliza Dijkstra para calcular el siguiente salto. Los anuncios remotos incluyen una edad máxima para retirar información obsoleta. Esta organización refleja el principio de estado de enlace usado por protocolos como OSPF (Moy, 1998).

4.5 Distance Vector Routing

DVR inicia únicamente con el costo de los enlaces directos. Al recibir el vector de un vecino, evalúa si llegar a cada destino por ese vecino mejora la tabla y conserva la alternativa de menor costo. Los empates se resuelven por identificador del siguiente salto. Las actualizaciones periódicas y disparadas usan split horizon con poison reverse, y un valor finito de infinito limita el conteo. Los vectores antiguos se descartan por secuencia y tiempo (Malkin, 1998).

4.6 Forwarding y routing concurrentes

AsyncNode mantiene dos colas. Forwarding procesa mensajes de usuario, entrega los dirigidos al nodo local y reenvía los restantes. Routing recibe paquetes info, instala LSP o vectores y emite anuncios periódicos. TcpTransport abre un servidor por nodo y transmite una trama JSON por conexión. El receptor valida que el emisor sea un vecino antes de aceptar el paquete; los fallos de conexión se registran sin detener las demás tareas.

5 Pruebas individuales

Las pruebas se ejecutaron el 7 de septiembre de 2026 sobre el commit e5e431812593a6ed3ee9f3c34fd63cfba6eb6425. Se utilizó la topología ponderada de nueve nodos incluida en examples/topo-demo.txt, basada en el mapa del enunciado. En cada modo se envió el texto Prueba individual desde A hasta H con TTL 16.

Topología ponderada usada en la simulación

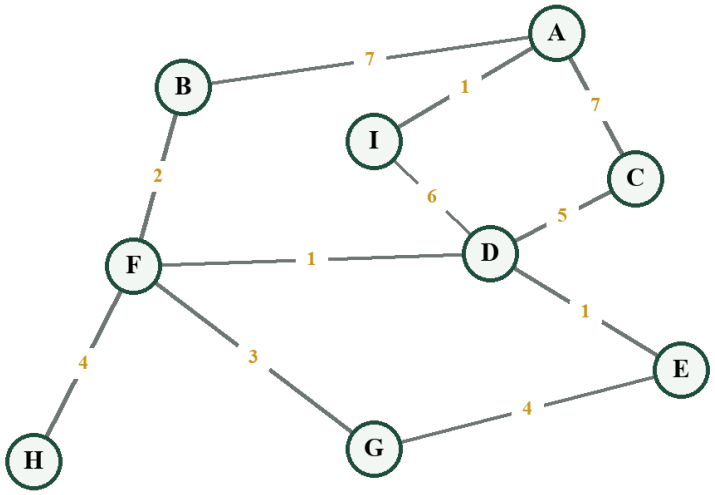


Figura 2. Topología de nueve nodos usada en las simulaciones.

5.1 Estrategia de verificación

Nivel	Comprobaciones	Resultado
Unidad	Paquetes, TTL, configuración y cálculo de cada algoritmo.	Aprobado
Convergencia	LSR y DVR generan las mismas rutas y costos que Dijkstra.	Aprobado
Fallo de enlace	La ruta A-B-C cambia de costo 2 a la alternativa A-C de costo 10.	Aprobado
Integración TCP	Tres nodos independientes A-B-C reenvían mensaje TCP hasta C.	Aprobado

Tabla 3. Cobertura de la validación automatizada.

5.2 Resultados de enrutamiento

Algoritmo	Eventos de control	Transmisiones	Ruta entregada
Dijkstra	0	4	A-I-D-F-H
Flooding	0	9	A-B-F-H
LSR	126	4	A-I-D-F-H
DVR	119	4	A-I-D-F-H

Tabla 4. Resultado observado al enviar un mensaje de A hacia H.

Los cuatro modos entregaron el payload sin agotar el TTL. Dijkstra, LSR y DVR eligieron A-I-D-F-H, con costo total 12. Flooding entregó primero por A-B-F-H, con costo 13, y necesitó nueve transmisiones; además, dos copias fueron descartadas como duplicadas en D. Los eventos de control de LSR y DVR corresponden a la distribución inicial necesaria para estabilizar las tablas.

```
PS> python -m unittest discover -s tests -v
test_three_independent_tcp_nodes_forward_message ... ok
test_every_algorithm_delivers ..................... ok
test_lsr_and_dvr_converge_to_dijkstra_costs ....... ok
test_routes_adapt_after_link_failure .............. ok

Ran 21 tests in 0.086s
OK
```

Figura 3. Resultado de la suite automatizada.

5.3 Adaptación ante cambios

Una prueba adicional empleó A-B-C con costos unitarios y un enlace directo A-C de costo 10. Antes del fallo, Dijkstra, LSR y DVR seleccionaron A-B-C con costo 2. Después de retirar B-C y volver a converger, los tres utilizaron A-C con costo 10. El resultado comprueba que la tabla no queda ligada a la configuración inicial y que los algoritmos dinámicos propagan el cambio.

5.4 Comunicación TCP

La prueba de integración levantó tres servidores AsyncNode en puertos locales diferentes. A envió el mensaje mensaje TCP hacia C; B lo recibió como vecino, consultó su tabla y lo reenvió; C lo colocó en su cola de entrega conservando a A como origen. La finalización ordenada de los tres servidores confirmó que las tareas pueden iniciarse y cancelarse sin dejar procesos pendientes.

6 Discusión

Los resultados muestran una diferencia clara entre el costo de datos y el costo de control. Dijkstra puro reenvía por una ruta óptima sin intercambiar anuncios porque parte de la topología completa, pero esa ventaja depende de información estática. LSR alcanza la misma tabla después de distribuir los LSP, mientras DVR converge intercambiando distancias con vecinos. En una ejecución prolongada, esos mensajes adicionales son el precio de adaptarse a cambios sin consultar el archivo global.

Flooding fue funcional, pero produjo más transmisiones y encontró una ruta de costo 13 en lugar de la ruta mínima de costo 12. El algoritmo prioriza difusión y robustez de alcance, no optimización de costo. La caché de duplicados y el TTL fueron indispensables: sin ambos controles, una topología con ciclos mantendría copias circulando y aumentaría rápidamente el tráfico.

LSR y DVR coincidieron con Dijkstra para todos los destinos de la topología de prueba. Sin embargo, su información interna es distinta. LSR conserva una base de enlaces y calcula

localmente el grafo; DVR nunca necesita el grafo completo y decide a partir de vectores. Poison reverse reduce ciclos de dos nodos, aunque no elimina todos los escenarios de convergencia lenta en redes mayores.

La prueba TCP demuestra separación entre algoritmo y transporte, pero no sustituye la validación de interoperabilidad en XMPP. En clase será necesario comprobar autenticación, nombres oficiales, formato de mensajes entre implementaciones de distintos estudiantes, latencia real y recuperación cuando un usuario se desconecte. Esos resultados deben agregarse únicamente después de observarlos.

7 Conclusiones

- La implementación modular permite ejecutar Dijkstra, Flooding, LSR y DVR con el mismo contrato de paquete y sin mezclar el algoritmo con el transporte.
- Las 21 pruebas automatizadas aprobaron la serialización, el TTL, la configuración, los cuatro algoritmos, la convergencia, el cambio de enlaces y el reenvío TCP.
- Dijkstra, LSR y DVR entregaron A-H por la ruta mínima de costo 12; Flooding también entregó, pero requirió más transmisiones y siguió una ruta de costo 13.
- LSR y DVR respetan el conocimiento local inicial y aprendieron las rutas mediante anuncios, por lo que no resuelven estáticamente la topología desde el archivo.
- La reacción ante la eliminación de B-C confirmó que las tablas pueden abandonar una ruta inválida y seleccionar la alternativa disponible.

8 Comentarios y trabajo pendiente

La parte individual queda preparada para la prueba de clase. Durante esa sesión se debe sustituir o adaptar TcpTransport al servidor XMPP, cargar el archivo oficial de nombres, comprobar mensajes con otros nodos y registrar evidencia de estabilización, rutas, latencias, caídas y reconexiones. El presente informe no asigna integrantes ni anticipa esos resultados.

9 Reproducibilidad

La implementación no requiere dependencias externas de ejecución. Desde la raíz del repositorio se pueden repetir las verificaciones con:

```
$env:PYTHONPATH = "src"
python -m unittest discover -s tests -v
python -m routing_lab simulate --algorithm lsr --topology examples/topo-
demo.txt --source A --destination H --message "Prueba individual"
```

10 Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to Algorithms. Cuarta edición. MIT Press.

Malkin, G. (1998). RIP Version 2. RFC 2453. RFC Editor. https://www.rfc-editor.org/rfc/rfc2453

Moy, J. (1998). OSPF Version 2. RFC 2328. RFC Editor. https://www.rfc-editor.org/rfc/rfc2328

Universidad del Valle de Guatemala. (2026). Laboratorio 3 Algoritmos de Enrutamiento. CC3067 Redes.

Marroquin, I. (2026). lab3_redes. Repositorio de código. https://github.com/Ihan-Marroquin/lab3_redes

Python Software Foundation. (s. f.). asyncio Asynchronous I O. https://docs.python.org/3/library/asyncio.html